commons-cli Fragment Translation Examples

This document samples the `commons-cli` fragment run from `commons-cli_deepseek-chat_body.log` and `data/analysis/commons_cli_evosuite_fragment.log`. The first section shows representative failure modes and a manually validated minimal Cangjie standard answer. The second section shows cases where compiler feedback helped the model reach a compiling final attempt.

Validation note: the standard-answer snippets were checked in a minimal Cangjie `cjpm` package under `/tmp/commons_cli_examples`; both probe packages built with `cjpm build success`.

Representative Failure Cases With Standard Answers

1. Option.type__

- Run status: `fail`; first errors: `compile 1/2 ✗ error: mismatched types; compile 2/2 ✗ error: mismatched types`
- Reasoning: 错在把 Java 的 `String.class` 当成普通值 `String` 翻译了；这里目标字段需要的是 `TypeInfo`，应使用 `TypeInfo.of<String>()` 这类形式。

Java source:

```
private Class<?> type__ = String.class;
```

Cangjie first attempt:

```
var type__: TypeInfo = String
```

Cangjie final model attempt:

```
var type__: TypeInfo = String
```

Cangjie standard-answer version:

```
static let STRING_VALUE: TypeInfo = TypeInfo.of<String>()
```

2. Option.equals

- Run status: `fail`; first errors: `compile 1/4 ✗ error: invalid binary operator '==' on type 'This' and 'Interface-Any'; compile 2/4 ✗ error: expected expression after '==', found '='`

- Reasoning: 错在直接照搬 Java 的对象同一性判断和 `instanceof`/强转结构；当前骨架里的 `this == obj`、`===` 和 `as` 都不是可用的等价替换。

Java source:

```
@Override
    public boolean equals(final Object obj) {
        if (this == obj) {
            return true;
        }
        if (!(obj instanceof Option)) {
            return false;
        }
        final Option other = (Option) obj;
        return Objects.equals(longOption, other.longOption) &&
Objects.equals(option, other.option);
    }
```

Cangjie first attempt:

```
public open func equals(obj: Object): Bool {
        if (this == obj) {
            return true
        }
        if (!(obj is Option)) {
            return false
        }
        let other = obj as Option
        return Objects.equals(longOption, other.longOption) &&
Objects.equals(option, other.option)
    }
```

Cangjie final model attempt:

```
public open func equals(obj: Object): Bool {
        if (this === obj) {
            return true
        }
        if (!(obj is Option)) {
            return false
        }
        let other = obj as Option
        return this.longOption == other.longOption && this.option == other.option
    }
```

Cangjie standard-answer version:

```
public func sameKey(other: Option): Bool {
    return this.option == other.option && this.longOption == other.longOption
}
```

3. Option.getKey

- Run status: `fail`; first errors: `compile 1/4 ✕ error: undeclared identifier 'null'; compile 2/4 ✕ error: undeclared identifier 'null'`
- Reasoning: 错在保留了 Java 的 `null` 判断；Cangjie 骨架里这些字段是非可空 `String`，不能再用 `null` 作为分支条件。

Java source:

```
String getKey() {
        return option == null ? longOption : option;
    }
```

Cangjie first attempt:

```
public open func getKey(): String {
        if (option == null) {
            return longOption
        } else {
            return option
        }
    }
```

Cangjie final model attempt:

```
public open func getKey(): String {
        if (option == null) {
            return longOption
        } else {
            return option
        }
    }
```

Cangjie standard-answer version:

```
public func getKey(): String {
    return if (option.isEmpty()) { longOption } else { option }
}
```

4. Option.hashCode

- Run status: `fail`; first errors: `compile 1/4 ✗ error: undeclared identifier 'Objects';  compile 2/4 ✗ error: expected '.', '(', '[', '{' or '?' after '?'`
- Reasoning: 错在依赖 Java 的 `Objects.hash` 和可空判断；当前字段是具体的非空字符串，应直接基于 Cangjie 字符串哈希组合。

Java source:

```
@Override
    public int hashCode() {
        return Objects.hash(longOption, option);
    }
```

Cangjie first attempt:

```
public open func hashCode(): Int32 { return Objects.hash(longOption, option) }
```

Cangjie final model attempt:

```
public open func hashCode(): Int32 {
        var hash: Int32 = 1
        if (longOption != null) {
            hash = 31 * hash + longOption.hashCode()
        }
        if (option != null) {
            hash = 31 * hash + option.hashCode()
        }
        return hash
    }
```

Cangjie standard-answer version:

```
public func stableHash(): Int64 {
    var hash: Int64 = 17
    hash = hash * 31 + option.hashCode()
    hash = hash * 31 + longOption.hashCode()
    return hash
}
```

5. OptionGroup.toString

- Run status: `fail`; first errors: `compile 1/4 ✗ error: 'hasNext' is not a member of class 'Iterator<Class-Option>'; compile 2/4 ✗ error: 'hasNext' is not a member of class`

`'Iterator<Class-Option>'`

- Reasoning: 错在逐字翻译 Java 的迭代器协议；Cangjie 这里不能用 `hasNext()/next()`，应改用 `for-in` 或本地可用的迭代 API。

Java source:

```
@Override
    public String toString() {
        final StringBuilder buff = new StringBuilder();

        final Iterator<Option> iter = getOptions().iterator();

        buff.append("[");

        while (iter.hasNext()) {
            final Option option = iter.next();

            if (option.getOpt() != null) {
                buff.append("-");
                buff.append(option.getOpt());
            } else {
                buff.append("--");
                buff.append(option.getLongOpt());
            }

            if (option.getDescription() != null) {
                buff.append(" ");
                buff.append(option.getDescription());
            }

            if (iter.hasNext()) {
                buff.append(", ");
            }
        }

        buff.append("]");

        return buff.toString();
    }
```

Cangjie first attempt:

```
public open func toString(): String {
    let buff = StringBuilder()
    let iter = getOptions().iterator()
    buff.append("[")
    while (iter.hasNext()) {
        let option = iter.next()
        if (option.getOpt() != "") {
            buff.append("-")
```

```
            buff.append(option.getOpt())
        } else {
            buff.append("--")
            buff.append(option.getLongOpt())
        }
        if (option.getDescription() != "") {
            buff.append(" ")
            buff.append(option.getDescription())
        }
        if (iter.hasNext()) {
            buff.append(", ")
        }
    }
    buff.append("]")
    return buff.toString()
}
```

Cangjie final model attempt:

```
public open func toString(): String {
    let buff = StringBuilder()
    let iter = getOptions().iterator()
    buff.append("[")
    while (iter.hasNext()) {
        let option = iter.next()
        if (option.option != "") {
            buff.append("-")
            buff.append(option.option)
        } else {
            buff.append("--")
            buff.append(option.longOption)
        }
        if (option.description != "") {
            buff.append(" ")
            buff.append(option.description)
        }
        if (iter.hasNext()) {
            buff.append(", ")
        }
    }
    buff.append("]")
    return buff.toString()
}
```

Cangjie standard-answer version:

```
public func joinOptionKeys(options: ArrayList<Option>): String {
    let buff = StringBuilder()
    buff.append("[")
```

```
    var first = true
    for (opt in options) {
        if (!first) { buff.append(", ") }
        buff.append(opt.key)
        first = false
    }
    buff.append("]")
    return buff.toString()
}
```

6. Options.addOption0

- Run status: `fail`; first errors: `compile 1/4 ✕ error: 'put' is not a member of class 'HashMap<Struct-String, Class-Option>'; compile 2/4 ✕ error: 'contains' is not a member of class 'ArrayList<Interface-Any>'`
- Reasoning: 错在继续使用 Java 集合方法名（如 `put`、`contains`、`append`）；这些方法和当前生成骨架中的 Cangjie 集合 API 不匹配。

Java source:

```
public Options addOption0(final Option opt) {
        final String key = opt.getKey();

        if (opt.hasLongOpt()) {
            longOpts.put(opt.getLongOpt(), opt);
        }

        if (opt.isRequired()) {
            if (requiredOpts.contains(key)) {
                requiredOpts.remove(requiredOpts.indexOf(key));
            }
            requiredOpts.add(key);
        }

        shortOpts.put(key, opt);

        return this;
    }
```

Cangjie first attempt:

```
public open func addOption0(opt: Option): Options {
        let key = opt.getKey()
        if (opt.hasLongOpt()) {
            longOpts.put(opt.getLongOpt(), opt)
        }
        if (opt.isRequired()) {
            if (requiredOpts.contains(key)) {
```

```
                requiredOpts.remove(requiredOpts.indexOf(key))
            }
            requiredOpts.append(key)
        }
        shortOpts.put(key, opt)
        return this
    }
```

Cangjie final model attempt:

```
public open func addOption0(opt: Option): Options {
        let key = opt.getKey()
        if (opt.hasLongOpt()) {
            longOpts[opt.getLongOpt()] = opt
        }
        if (opt.isRequired()) {
            if (requiredOpts.contains(key)) {
                requiredOpts.remove(requiredOpts.indexOf(key))
            }
            requiredOpts.append(key)
        }
        shortOpts[key] = opt
        return this
    }
```

Cangjie standard-answer version:

```
public func addOption0(opt: Option): Options {
    let key = opt.getKey()
    if (!opt.longOption.isEmpty()) { longOpts.add(opt.longOption, opt) }
    if (!key.isEmpty()) { shortOpts.add(key, opt) }
    return this
}
```

7. Options.toString

- Run status: `fail`; first errors: `compile 1/4 ✕ error: 'toString' is not a member of class 'HashMap<Struct-String, Class-Option>'; compile 2/4 ✕ error: 'toString' is not a member of class 'HashMap<Struct-String, Class-Option>'`
- Reasoning: 错在假设 Cangjie 的 `HashMap` 也有 Java 式 `toString()`；这里需要显式遍历 map 并手动拼接字符串。

Java source:

```
@Override
    public String toString() {
```

```
        final StringBuilder buf = new StringBuilder();

        buf.append("[ Options: [ short ");
        buf.append(shortOpts.toString());
        buf.append(" ] [ long ");
        buf.append(longOpts);
        buf.append(" ]");

        return buf.toString();
    }
```

Cangjie first attempt:

```
public open func toString(): String {
    let buf = StringBuilder()
    buf.append("[ Options: [ short ")
    buf.append(shortOpts.toString())
    buf.append(" ] [ long ")
    buf.append(longOpts.toString())
    buf.append(" ]")
    return buf.toString()
}
```

Cangjie final model attempt:

```
public open func toString(): String {
    let buf = StringBuilder()
    buf.append("[ Options: [ short ")
    buf.append(shortOpts.toString())
    buf.append(" ] [ long ")
    buf.append(longOpts.toString())
    buf.append(" ]")
    return buf.toString()
}
```

Cangjie standard-answer version:

```
public func describeMap(shortOpts: HashMap<String, Option>): String {
    let buff = StringBuilder()
    buff.append("[short ")
    var first = true
    for ((key, _) in shortOpts) {
        if (!first) { buff.append(",") }
        buff.append(key)
        first = false
    }
    buff.append("]")
```

```
        return buff.toString()
    }
```

8. PatternOptionBuilder.STRING_VALUE

- Run status: `fail`; first errors: `compile 1/2 ✗ the 'method' field in JSON output is empty; compile 1/2 ✗ error: expected a member name after '.' in qualified name, found keyword 'type'`
- Reasoning: 错在输出了类级别代码或非法的 `String.type`；该 fragment 只需要字段初始化表达式，类字面量应翻译为 `TypeInfo.of<String>()`。

Java source:

```
public static final Class<String> STRING_VALUE = String.class;
```

Cangjie first attempt:

```
public open class PatternOptionBuilder {
    static let STRING_VALUE: TypeInfo = String.Type
}
```

Cangjie final model attempt:

```
static let STRING_VALUE: TypeInfo = String.type
```

Cangjie standard-answer version:

```
public static let STRING_VALUE: TypeInfo = TypeInfo.of<String>()
```

9. ParseException.ParseException

- Run status: `fail`; first errors: `compile 1/4 ✗ error: extra argument given for parameter list '()' in call; compile 2/4 ✗ error: extra argument given for parameter list '()' in call`
- Reasoning: 错在没有对齐当前生成骨架中的父类构造器形态；因此在这个上下文里直接调用 `super(message)` 是非法的。

Java source:

```
public ParseException(final String message) {
        super(message);
```

```
    }
```

Cangjie first attempt:

```
public init(message: String) {
    super(message)
}
```

Cangjie final model attempt:

```
public init(message: String) {
    super()
    this.message = message
}
```

Cangjie standard-answer version:

```
public class ParseException <: Exception {
    public init(message: String) {
        super(message)
    }
}
```

10. UnrecognizedOptionException.UnrecognizedOptionException

- Run status: `fail`; first errors: `compile 1/4 ✗ error: call to 'super' must be first expression in constructor of class 'UnrecognizedOptionException'; compile 2/4 ✗ error: call to 'super' must be first expression in constructor of class 'UnrecognizedOptionException'`
- Reasoning: 错在只看方法体文本，忽略了生成后的类结构和字段初始化上下文；最终构造器违反了 Cangjie 对父类构造调用顺序的要求。

Java source:

```
public UnrecognizedOptionException(final String message, final String
option_param) {
        super(message);
        this.option = option_param;
    }
```

Cangjie first attempt:

```
public init(message: String, option_param: String) {
        super(message)
        this.option = option_param
    }
```

Cangjie final model attempt:

```
public init(message: String, option_param: String) {
    super(message)
    this.option = option_param
}
```

Cangjie standard-answer version:

```
public class CliException <: Exception {
    public let option: String
    public init(message: String, option: String) {
        super(message)
        this.option = option
    }
}
```

Feedback Iterations That Succeeded

1. Option.clone

- Run status: `success`; first errors: `compile 1/4 ✕ error: undeclared type name 'CloneNotSupportedException'; compile 2/4 ✕ error: 'clone' is not a member of enum 'Option<Class-Object>'`
- Reasoning: 错在照搬 Java 的 `super.clone()` 和受检异常模式；Cangjie 这里需要用可用构造器和字段显式创建副本。

Java source:

```
@Override
    public Object clone() {
        try {
            final Option option = (Option) super.clone();
            option.values = new ArrayList<>(values);
            return option;
        } catch (final CloneNotSupportedException cnse) {
            throw new RuntimeException(
                    "A CloneNotSupportedException was thrown: " +
cnse.getMessage());
```

```
        }
    }
```

Cangjie first attempt:

```
public open func clone(): Any {
    try {
        let option = super.clone() as Option
        option.values = ArrayList<String>(values)
        return option
    } catch (e: CloneNotSupportedException) {
        throw RuntimeException("A CloneNotSupportedException was thrown: " +
e.getMessage())
    }
}
```

Cangjie final success code:

```
public open func clone(): Any {
    let option = Option()
    option.option = this.option
    option.values = ArrayList<String>(this.values)
    return option
}
```

2. Util.stripLeadingAndTrailingQuotes

- Run status: `success`; first errors: `compile 1/4 ✕ error: 'substring' is not a member of struct 'String'; compile 2/4 ✕ error: 'slice' is not a member of struct 'String'`
- Reasoning: 错在使用了 Java/Python 风格的字符串截取 API；可编译版本需要使用 Cangjie 的区间索引语法。

Java source:

```
static String stripLeadingAndTrailingQuotes(String str) {
        final int length = str.length();
        if (length > 1
                && str.startsWith("\"")
                && str.endsWith("\"")
                && str.substring(1, length - 1).indexOf('"') == -1) {
            str = str.substring(1, length - 1);
        }

        return str;
    }
```

Cangjie first attempt:

```
static func stripLeadingAndTrailingQuotes(str: String): String {
        let length = str.length
        if (length > 1 && str.startsWith("\"") && str.endsWith("\"") &&
str.substring(1, length - 1).indexOf('"') == -1) {
            str = str.substring(1, length - 1)
        }
        return str
    }
```

Cangjie final success code:

```
static func stripLeadingAndTrailingQuotes(str: String): String {
        let length = str.size
        if (length > 1 && str.startsWith("\"") && str.endsWith("\"") &&
str[1..length-1].indexOf('"') == -1) {
            return str[1..length-1]
        }
        return str
    }
```

3. Util.stripLeadingHyphens

- Run status: `success`; first errors: `compile 1/4 ✕ error: undeclared identifier 'null'; compile 2/4 ✕ error: expected operator or end of expression, found ':'`
- Reasoning: 错在照搬 Java 的可空返回和 `substring` 调用；修复时改用空字符串作为本地哨兵值，并使用 Cangjie 区间索引。

Java source:

```
static String stripLeadingHyphens(final String str) {
        if (str == null) {
            return null;
        }
        if (str.startsWith("--")) {
            return str.substring(2);
        }
        if (str.startsWith("-")) {
            return str.substring(1);
        }

        return str;
    }
```

Cangjie first attempt:

```
static func stripLeadingHyphens(str: String): String {
        if (str == null) {
            return null
        }
        if (str.startsWith("--")) {
            return str.substring(2)
        }
        if (str.startsWith("-")) {
            return str.substring(1)
        }
        return str
    }
```

Cangjie final success code:

```
static func stripLeadingHyphens(str: String): String {
    if (str == "") {
        return ""
    }
    if (str.startsWith("--")) {
        return str[2..str.size]
    }
    if (str.startsWith("-")) {
        return str[1..str.size]
    }
    return str
}
```

4. Options.addOptionGroup

- Run status: `success`; first errors: `compile 1/4 ✕ error: 'append' is not a member of class 'ArrayList<Interface-Any>'; compile 2/4 ✕ error: 'append' is not a member of class 'ArrayList<Interface-Any>'`
- Reasoning: 错在使用了 Python/Java 风格的 list/map 修改方法；当前 Cangjie 骨架期望的是 `ArrayList.add` 和 map 下标赋值。

Java source:

```
public Options addOptionGroup(final OptionGroup group) {
        if (group.isRequired()) {
            requiredOpts.add(group);
        }

        for (final Option option : group.getOptions()) {
            option.setRequired(false);
            addOption0(option);
```

```
            optionGroups.put(option.getKey(), group);
        }

        return this;
    }
```

Cangjie first attempt:

```
public func addOptionGroup(group: OptionGroup): Options {
        if (group.required) {
            requiredOpts.append(group)
        }
        for (option in group.getOptions()) {
            option.required = false
            addOption0(option)
            optionGroups.put(option.getKey(), group)
        }
        return this
    }
```

Cangjie final success code:

```
public func addOptionGroup(group: OptionGroup): Options {
        if (group.required) {
            requiredOpts.add(group)
        }
        for (option in group.getOptions()) {
            option.required = false
            addOption0(option)
            optionGroups[option.getKey()] = group
        }
        return this
    }
```

5. PatternOptionBuilder.isValueCode

- Run status: `success`; first errors: `compile 1/4 ✕ error: invalid binary operator '==' on type 'Rune' and 'Struct-String'; compile 2/4 ✕ error: invalid binary operator '==' on type 'Rune' and 'Struct-String'`
- Reasoning: 错在把 `Rune` 参数和字符串字面量比较；正确做法是使用 Cangjie 的 rune 字面量，例如 `r'@'`。

Java source:

```
public static boolean isValueCode(final char ch) {
        return ch == '@' || ch == ':' || ch == '%' || ch == '+' || ch == '#' || ch
```

```
== '<'
                || ch == '>' || ch == '*' || ch == '/' || ch == '!';
    }
```

Cangjie first attempt:

```
public static func isValueCode(ch: Rune): Bool { return ch == '@' || ch == ':' ||
ch == '%' || ch == '+' || ch == '#' || ch == '<' || ch == '>' || ch == '*' || ch
== '/' || ch == '!' }
```

Cangjie final success code:

```
public static func isValueCode(ch: Rune): Bool {
        return ch == r'@' || ch == r':' || ch == r'%' || ch == r'+' || ch == r'#'
|| ch == r'<' || ch == r'>' || ch == r'*' || ch == r'/' || ch == r'!'
    }
```

Observed Error Classes

- Java reflection class literals such as `String.class` were repeatedly rendered as `String`, `String.type`, or `String.Type`; the compiling Cangjie form is `TypeInfo.of<String>()`.
- Java null checks cannot be copied directly when the skeleton type is non-optional `String`; the minimal repair is usually sentinel-based or requires making the type optional at the skeleton/type-resolution level.
- Java collection APIs were often copied literally (`put`, `contains`, `hasNext`, `toString`); Cangjie collections in these probes used `add`, `for-in`, indexing, and explicit StringBuilder rendering.
- Java constructor and exception patterns need Cangjie class design, not body-only rewriting; `super(...)` must fit the Cangjie superclass constructor and field initialization rules.
- Compiler feedback was effective for local syntax/API substitutions such as `Array` construction, string slicing, `ArrayList.add`, and rune literals, but less effective for semantic gaps such as reflection, nullability, and Java iterator protocols.